Mars Habitat Automation Platform

Event-Driven Resilience in a Hostile Environment

Project Submission
Laboratory of Advanced Programming 2025/2026

Deda Daniel
Pagliarini Paolo
Macrì Francesco
Torresi Flavio

NotebookLM

The Mission Briefing

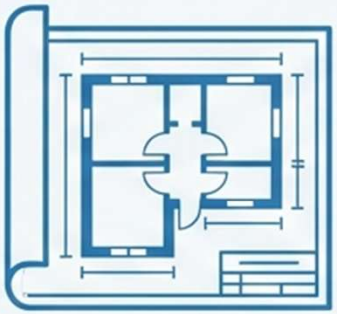
1 The Threat

- Thermodynamic failure due to a partially destroyed automation stack.
- The habitat is operating without a cohesive safety net.

2 The Chaos

Sensors speak incompatible dialects. Heterogeneous data streams consist of both synchronous REST polling (e.g., greenhouse_temperature) and asynchronous Telemetry via SSE (e.g., mars/telemetry/solar_array).

3 The Objective

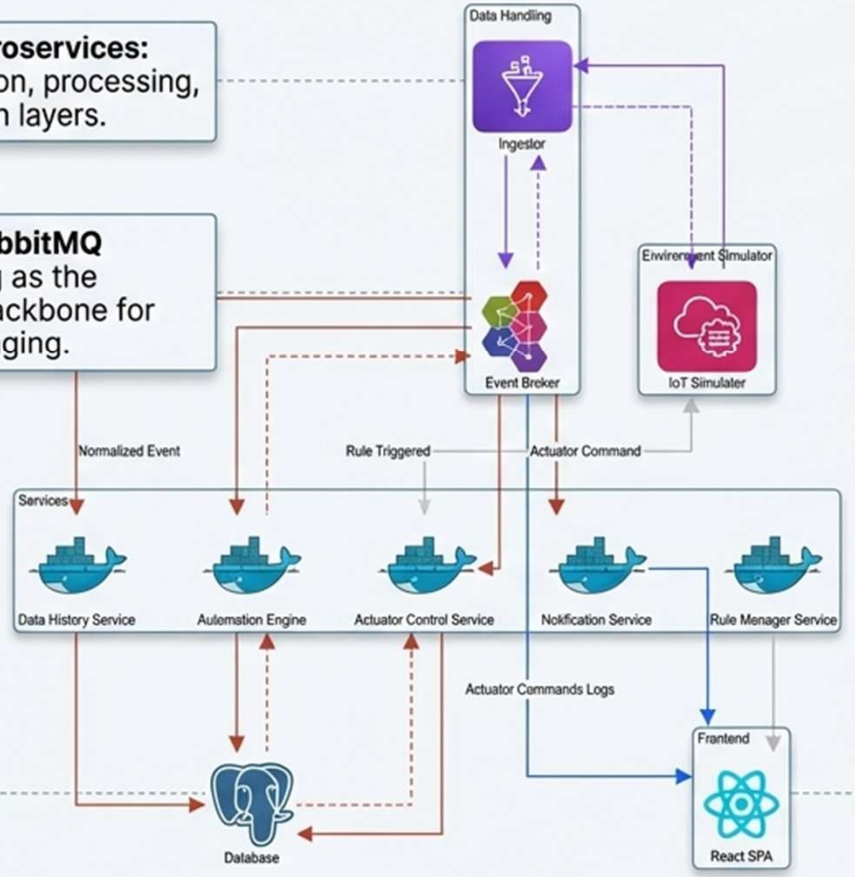


Build a distributed, event-driven platform to ingest heterogeneous data, normalize it, evaluate survival rules, and trigger physical actuators in real-time.

System Architecture Overview

Decoupled Microservices: Isolating ingestion, processing, and presentation layers.

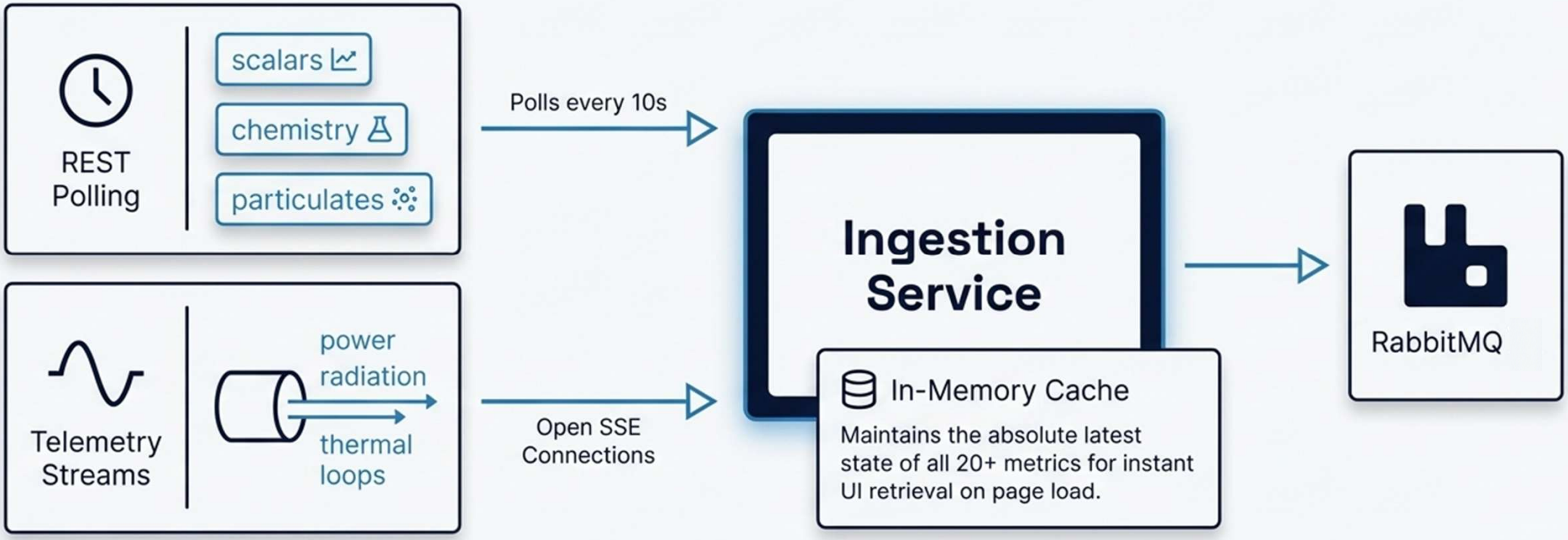
Centralized RabbitMQ Broker: Serving as the event-driven backbone for Pub/Sub messaging.



PostgreSQL: Providing persistent state and historical audit trails.

React SPA: Providing real-time human operator intervention and system observability.

Data Ingestion at the Edge



The Standard Internal Event Schema

Crucial design pattern: Downstream services (UI, rule engine) never need to parse specific device protocols. Everything speaks one language.

Before: Chaotic Nested JSON

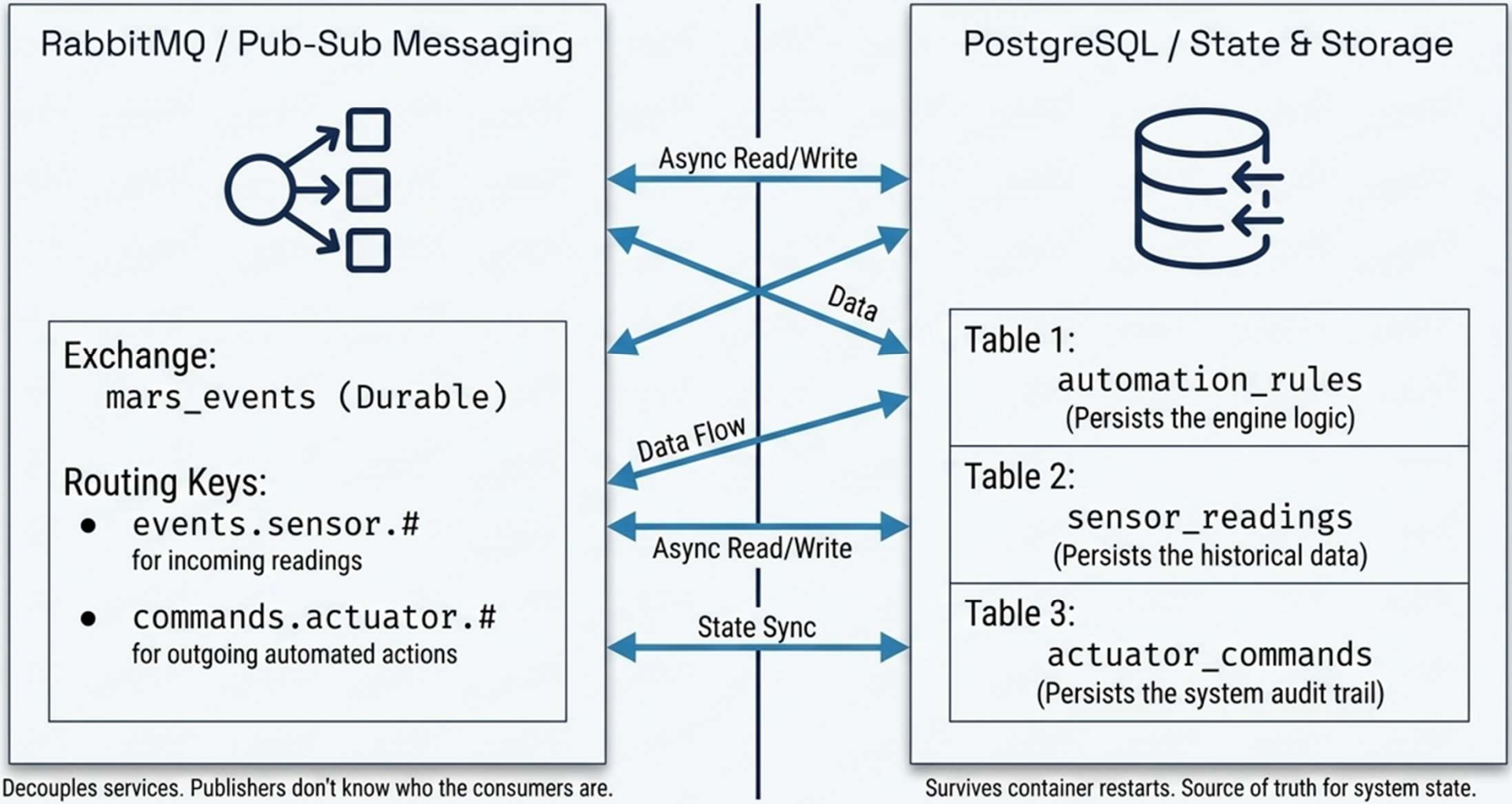
```
{
  "sensor": "air_quality_pm25",
  "measurements": [
    {"pm1": 12},
    {"pm2.5": 45},
    {"pm10": 80}
  ]
}
```

Ingestion Normalizer

After: Standardized Flat JSON

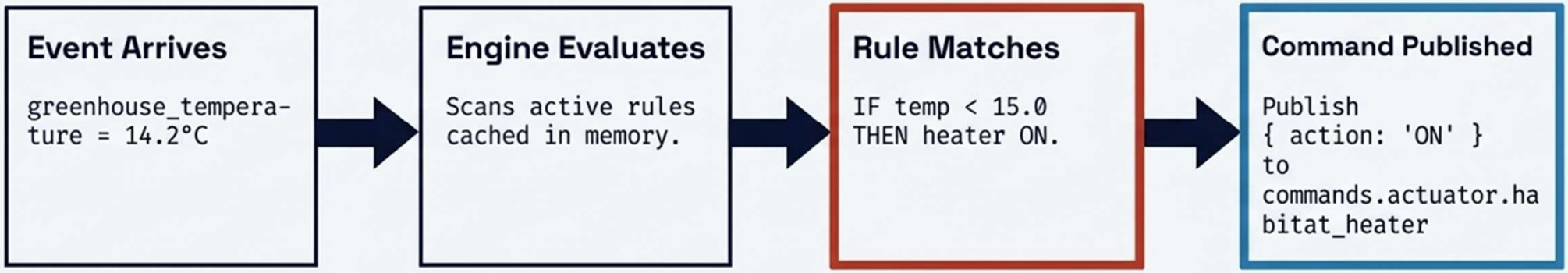
```
{
  "event_id": "8f72a-4b…",
  "sensor_id": "air_quality_pm25_pm25",
  "timestamp": "2036-03-05T14:20:00Z",
  "metric": "pm25_ug_m3",
  "value": 45,
  "unit": "ug/m3",
  "source": "rest",
  "status": "warning",
  "raw_schema": "rest.particulate.v1"
}
```

The Nervous System

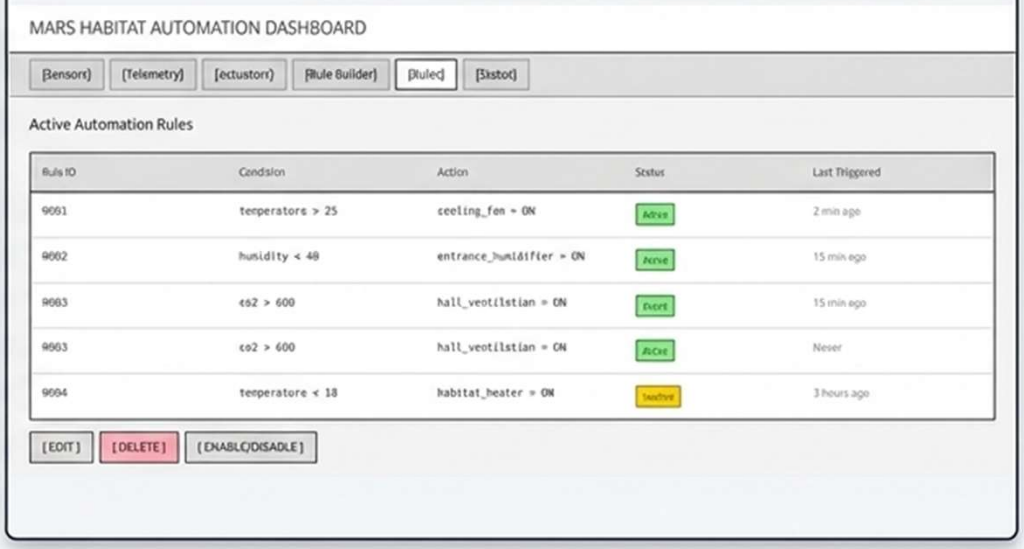


The Automation Engine

```
IF <sensor_name> <operator> <value>
THEN set <actuator_name> to ON | OFF
```



Rule Management & Persistence



- **REST CRUD APIs**

  `rule-manager-service` allows operators to create, edit, delete, or temporarily toggle rules.

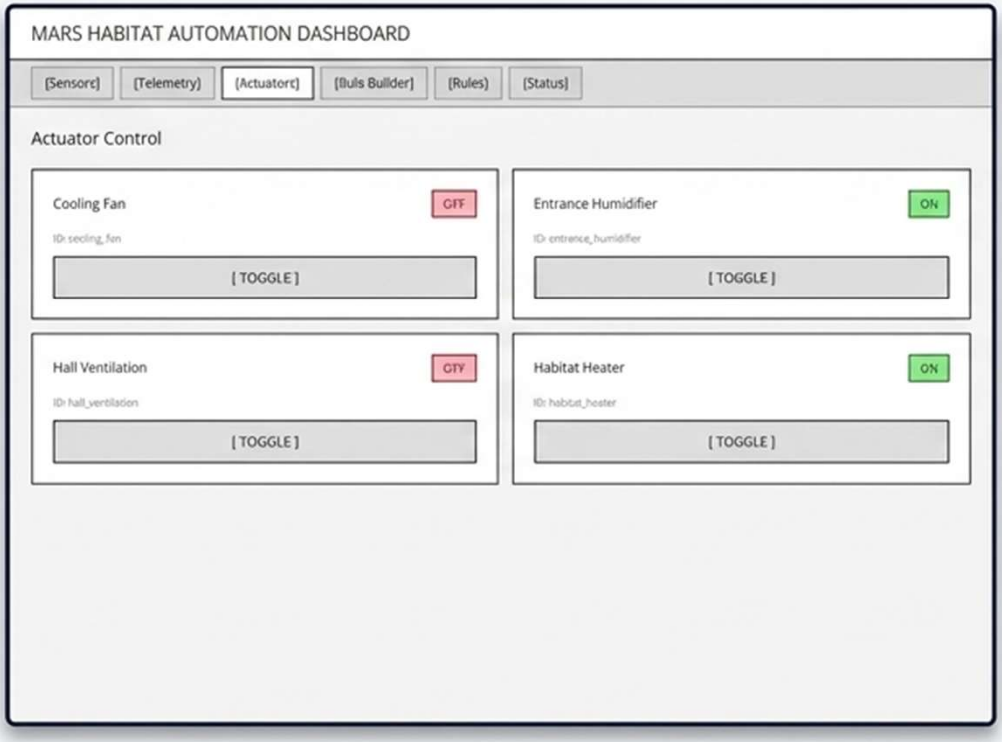
- **PostgreSQL Persistence**

  All rule changes are committed to the `automation_rules` table to survive container restarts.

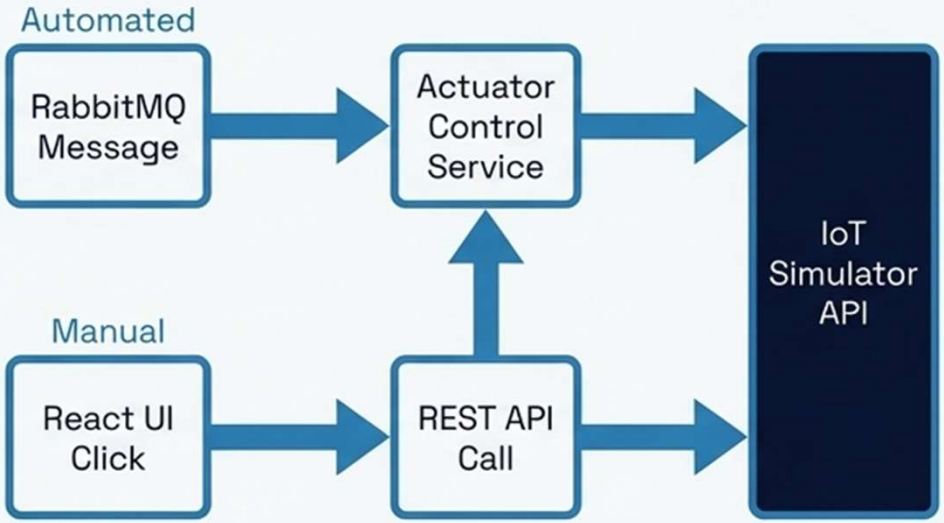
- **Memory Reloading**

  The engine periodically pulls updates from the database every 30 seconds (RULES_RELOAD_INTERVAL) to ensure lock-free, rapid evaluation.

Actuator Control



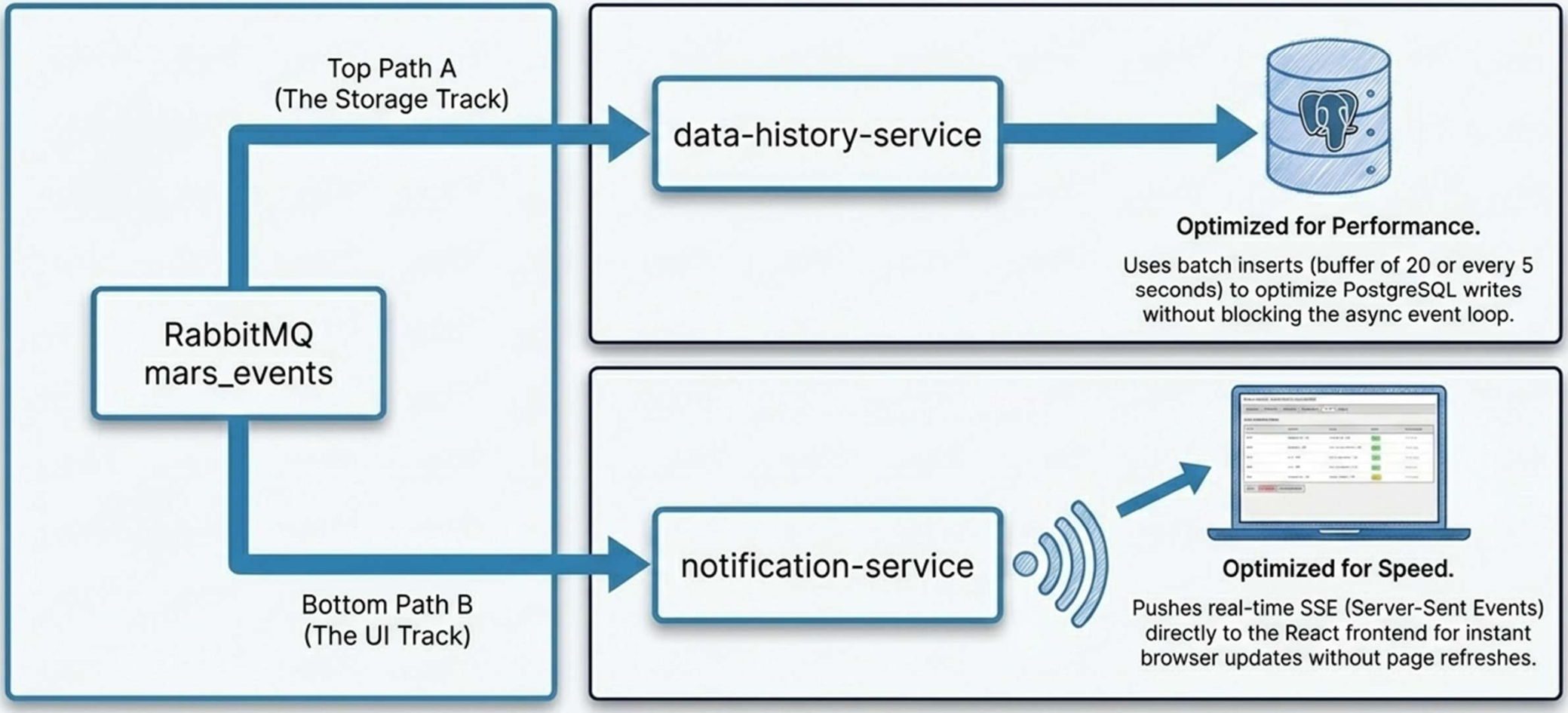
Dual Control Mechanics



Audit Trail

All state changes, whether manual or automated, are intercepted and logged to the PostgreSQL actuator_commands table for total accountability.

Data History & Real-Time Notifications



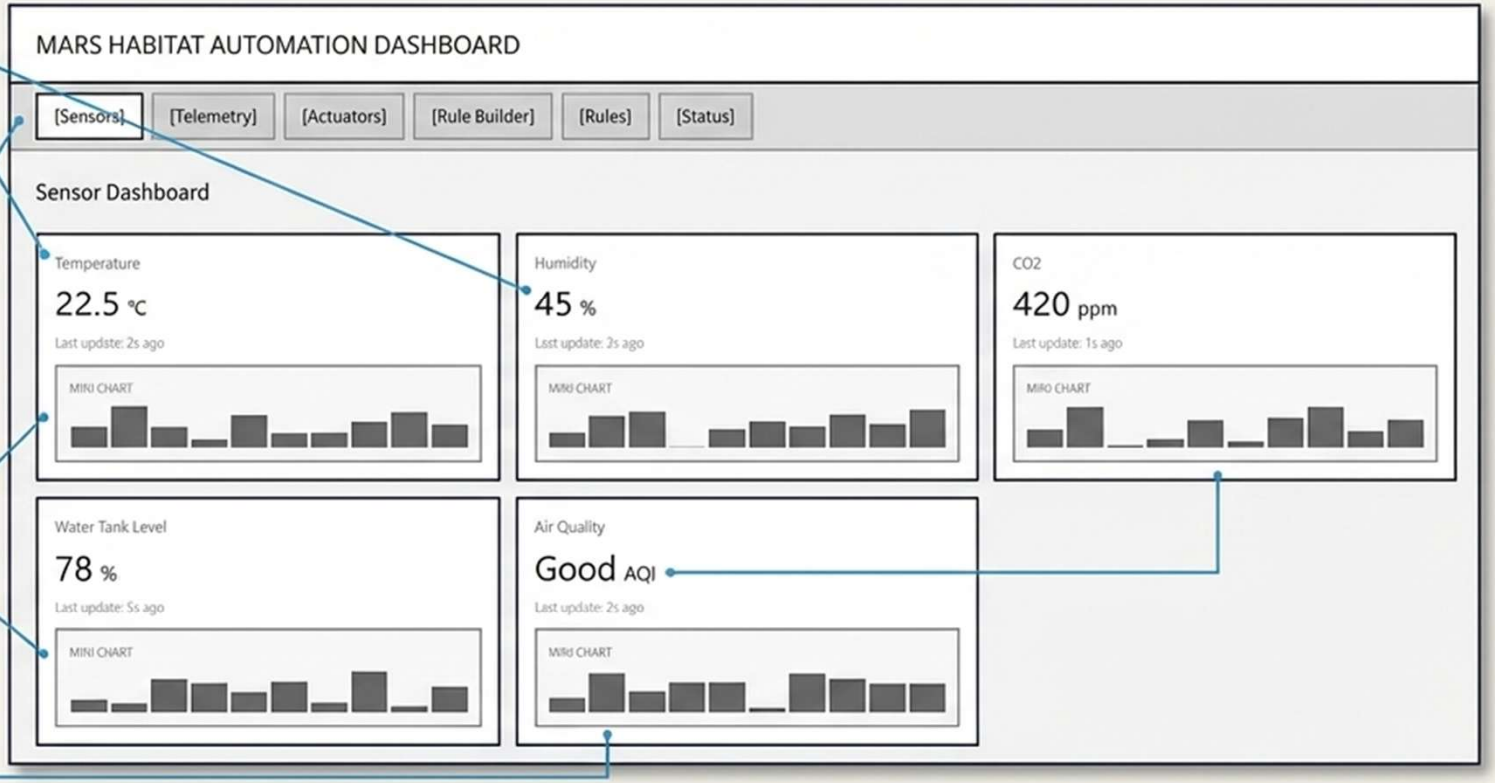
Operator View: Sensor Dashboard

Real-Time Updates: Values are pushed instantly via SSE.

Zero-Wait Rendering: In-memory backend caching populates the UI immediately on load.

Mini-Charts: Instant visual context for recent environmental trends.

Anomaly Detection: Status indicators flag dangerous conditions instantly.



Operator View: Live Telemetry

Handling Continuous Streams

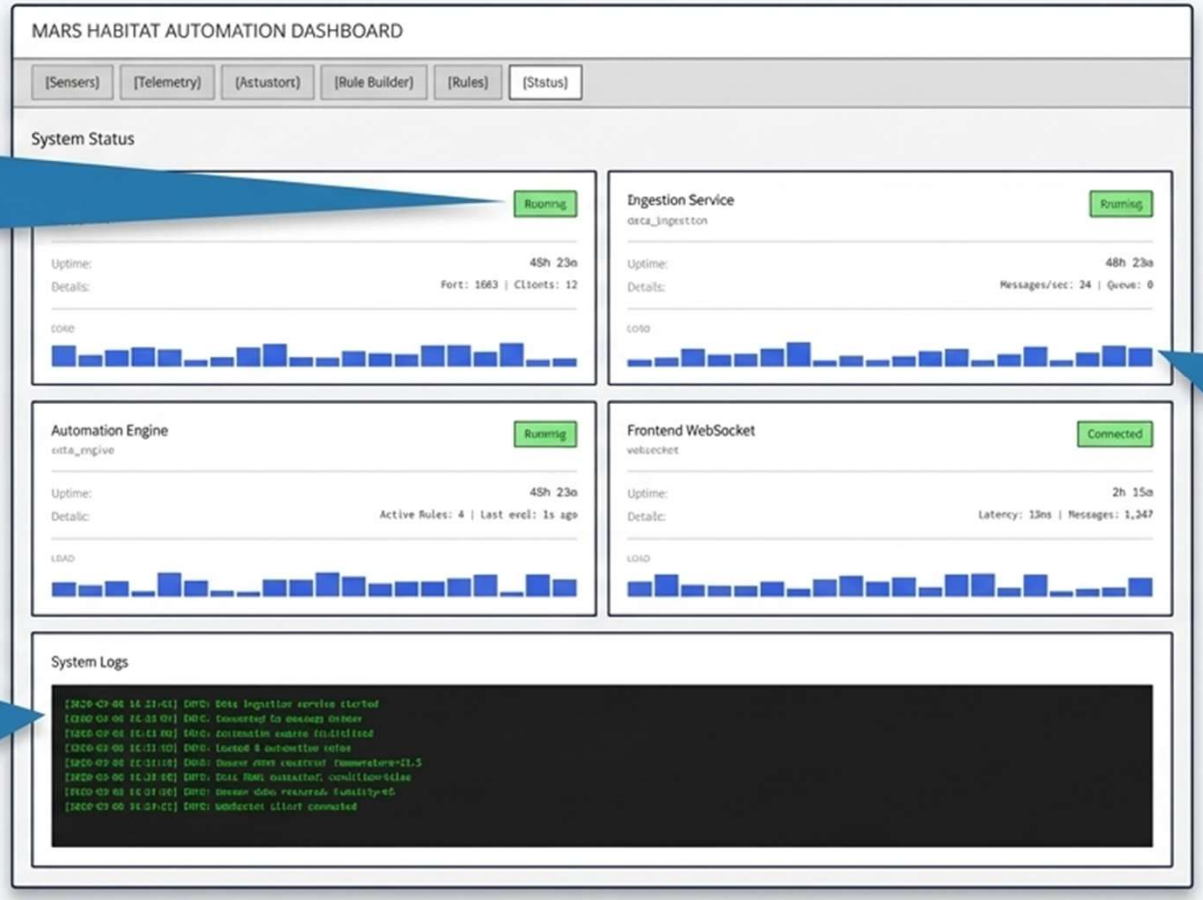
- Continuous time-series charts track the most critical subsystems (Solar Array, Radiation, Thermal Loop).
- **Seamless Data Blending:** The view is populated initially by the data-history aggregation API for immediate context.
- **Live Edge:** As the page remains open, the chart dynamically appending real-time SSE stream points to the leading edge of the graph.

System Status & Auditing

1

Health Monitoring:

Live health checks of critical microservices (Message Broker, Ingestion, Engine, WebSockets).



2

Throughput Metrics:

Instant visibility into uptime, message throughput rates, and active rule counts.

3

Live System Logs:

A real-time console streaming audit logs of automated behavior and rule triggers.

Tech Stack & Infrastructure

Backend APIs

Python & FastAPI

- Six decoupled, asynchronous microservices handling high-performance HTTP requests and data routing.

Messaging & State

RabbitMQ & PostgreSQL

- AMQP message broker for decoupled Pub/Sub event distribution, backed by robust relational persistence for rules and history.

Frontend Dashboard

React 18 & Tailwind CSS

- A responsive Single Page Application built with Vite, utilizing Recharts for data visualization and SSE for real-time reactivity.

Infrastructure

Fully Containerized

- Total reproducibility. A single `docker-compose up` launches the entire environment seamlessly, orchestrating all 10 containers.

Mission Accomplished

- ✔ Event-Driven Architecture Implemented (RabbitMQ Integration)
- ✔ Unified Event Schema Designed (Heterogeneous -> Standard JSON)
- ✔ Persistent Automation Engine Deployed (PostgreSQL Storage)
- ✔ Real-Time Operator Dashboard Built (React + SSE)
- ✔ 100% Docker Reproducibility Achieved (Single compose file)

Final Evaluation: All 20 requested user stories fulfilled.
The habitat is stabilized, observable, and fully automated.